

STM32 AND VS CODE

HOW TO CREATE A NEW PROJECT

Autor:
Rodrigo CASTILLEJOS

Proyecto:
Control de Dispositivos Embebidos

June 10, 2026

RESUMEN

Este documento técnico presenta una guía metodológica detallada para el diseño, configuración e implementación de un entorno de desarrollo de firmware embebido unificado, orientado a microcontroladores de la familia STM32 mediante el uso del editor de código Visual Studio Code en sistemas operativos Windows. El flujo de desarrollo presentado abarca desde la instalación inicial de las herramientas indispensables de software (Visual Studio Code, STM32CubeMX y CMake), hasta la integración práctica del diseño del hardware utilizando como plataforma física la tarjeta de desarrollo NUCLEO-F303RE.

Asimismo, se exponen las mejores prácticas de optimización y organización dentro de VS Code. Esto incluye la creación y activación de perfiles dedicados (Profiles) para el desarrollo con STM32, lo cual previene la saturación de recursos en el IDE, así como la asignación manual del ejecutable de CMake en las configuraciones globales. A través del uso de la extensión oficial de STMicroelectronics, se detalla la vinculación de la toolchain GCC y el depurador físico ST-Link a través de un GDB Server local para realizar la carga y monitoreo del firmware.

TABLA DE CONTENIDOS

1	Aplicaciones requeridas para la creación de proyectos STM32	4
1.1	Extensiones Recomendadas para Visual Studio Code	4
1.2	Descarga y Configuración de CMake	5
2	Configuraciones Previas en Visual Studio Code	7
2.1	Creación y Uso de Perfiles (Profiles)	7
2.1.1	¿Para qué sirven y cuáles son sus ventajas?	7
2.1.2	Pasos para crear un nuevo perfil	7
2.1.3	Activación del perfil	8
2.2	Configuración de la Ruta del Ejecutable de CMake	8
2.3	Configuración de la Ruta del Ejecutable de STM32CubeMX	9
3	Creación de un nuevo proyecto usando STM32CubeMX	11
3.1	Prueba de Funcionamiento: Parpadeo de LED (Blinky)	17
4	Integración de Controladores y Librerías Propias en CMake	22
4.1	Estructura del archivo CMakeLists.txt (Antes)	22
4.2	Modificación de Fuentes y Cabeceras (Después)	23
5	Configuración del Puerto Serie e Impresión de Valores Flotantes	24
5.1	Código de Prueba para el Sensor OPT3001	24
5.2	Uso de la Extensión Serial Monitor en Visual Studio Code	29
5.3	El problema de la impresión de Punto Flotante	29
5.4	Habilitación de Flotantes en el enlazador de CMake	29
5.5	Comparativa de Métodos para Habilitar Punto Flotante	30
6	Gestión de Múltiples Archivos main en un solo Proyecto	32
6.1	Método 1: Exclusión de archivos en STM32CubeIDE	32
6.2	Método 2: Compilación Condicional en VS Code mediante CMake	32
6.2.1	Paso A: Modificación en CMakeLists.txt principal	32
6.2.2	Paso B: Modificación crítica en el script autogenerado de CubeMX	33
6.3	Método 3: Truco Universal (Directivas del Preprocesador de C)	33
6.4	El problema visual de IntelliSense en los archivos inactivos	35
6.4.1	¿Por qué ocurre?	35

1 APLICACIONES REQUERIDAS PARA LA CREACIÓN DE PROYECTOS STM32

Para poder inicializar, configurar y desarrollar un nuevo proyecto destinado a tarjetas de desarrollo o microcontroladores STM en Visual Studio Code, es fundamental descargar e instalar las siguientes herramientas en su versión para Windows:

VISUAL STUDIO CODE PARA WINDOWS: El entorno de desarrollo (IDE) ligero donde se escribirá, compilará y depurará el código C/C++. Se recomienda instalar posteriormente las extensiones oficiales de STMicroelectronics (como *STM32 VS Code Extension*).

- Enlace de descarga: code.visualstudio.com

STM32CUBEMX: Herramienta gráfica oficial de STMicroelectronics que se utiliza para configurar los periféricos del microcontrolador (pines GPIO, relojes del sistema, puertos USART, etc.) y generar el código de inicialización base del proyecto.

- Enlace de descarga: st.com/stm32cubemx

CMAKE: El sistema de generación de compilación multiplataforma. Las nuevas plantillas de proyectos de STM32CubeMX utilizan CMake para gestionar y organizar la compilación de todos los archivos de código fuente.

- Enlace de descarga: cmake.org

Variable de Entorno (PATH)

Durante la instalación de **CMake**, asegúrese de marcar la opción que dice "**Add CMake to the system PATH for all users**" (o para el usuario actual). Esto es indispensable para que Visual Studio Code pueda localizar y ejecutar CMake de forma automática desde la terminal.

1.1 Extensiones Recomendadas para Visual Studio Code

Una vez instalado Visual Studio Code, es necesario equiparlo con las extensiones adecuadas para habilitar el soporte de desarrollo y depuración de microcontroladores. Abra el panel de Extensiones en VS Code mediante el atajo `Ctrl` + `Shift` + `X` e instale las siguientes herramientas:

- **C/C++ Extension Pack:** Proporciona soporte esencial para el lenguaje, incluyendo el autocompletado inteligente (IntelliSense), formateo automático, navegación por símbolos y depuración básica.
- **CMake Tools:** Permite a VS Code interpretar, configurar y compilar proyectos que utilicen sistemas de archivos basados en CMake de forma gráfica e integrada.
- **Serial Monitor:** Extensión oficial de Microsoft que integra un monitor de terminal serie directamente en las pestañas del editor para interactuar con los datos enviados por la tarjeta STM32.
- **STM32CubeIDE for Visual Studio Code:** La extensión principal de STMicroelectronics. Al instalar esta extensión, se instalará de forma automática el paquete completo que incluye 15 extensiones integradas, entre las que destacan:
 - *STM32Cube Core* (Núcleo de soporte básico)
 - *STM32Cube Bundles Manager* (Gestor de recursos y librerías)
 - *STM32Cube CMake Support* (Integración y soporte para compilación basada en CMake)
 - *STM32Cube Project Manager* (Importación y creación de proyectos desde CubeMX)

- *STM32Cube Build Analyzer* (Analizador visual del resultado de compilación y uso de memoria)
- *STM32Cube Debug Core* (Motor de depuración de firmware)
- *STM32Cube Debug Generic GDB Server* (Servidor GDB para depuración genérica)
- *STM32Cube Debug STLink GDB Server* (Servidor GDB específico para depuradores físicos ST-Link de ST)
- *STM32Cube Debug Jlink GDB Server* (Servidor GDB específico para depuradores Segger J-Link)
- *STM32Cube RTOS Views* (Vistas de depuración en tiempo real para sistemas operativos RTOS)
- *STM32Cube Registers View* (Visor detallado del estado de los registros SFR del microcontrolador)
- *STM32Cube clangd* (Servicio de lenguaje avanzado basado en Clang para análisis y refactorización)
- *Memory Inspector* (Herramienta gráfica para evaluar las secciones de RAM y Flash del chip)
- *Web and Desktop Serial Monitor* (Monitor serie alternativo y avanzado para tramas de comunicación)

Instalación del Paquete de STMicroelectronics

Al instalar la extensión **STM32CubeIDE for Visual Studio Code**, VS Code descargará e instalará automáticamente en segundo plano todas las demás extensiones asociadas de STMicroelectronics. No es necesario buscarlas ni instalarlas individualmente.

Una vez finalizada la instalación de estas herramientas, observará que en la **barra de actividad lateral izquierda** de Visual Studio Code aparecerán dos nuevos iconos dedicados:

1. El icono de **CMake** (representado por una figura geométrica tridimensional), desde el cual podrá configurar y compilar objetivos.
2. El logotipo oficial de **STMicroelectronics**, el cual da acceso directo a la gestión del proyecto, visualización de registros y herramientas de análisis del microcontrolador.

1.2 Descarga y Configuración de CMake

Para realizar la descarga e instalación de CMake en su sistema, siga los siguientes pasos:

1. Ingrese a la página oficial de descargas de CMake a través del enlace proporcionado anteriormente.
2. Al cargar el portal, identificará que las descargas se dividen en dos grandes opciones:
 - **Source Distributions:** Bloques que contienen únicamente el código fuente original del programa, recomendados si se desea compilar la herramienta desde cero.
 - **Binary Distributions:** Paquetes con ejecutables precompilados y listos para ser utilizados directamente en el sistema operativo.
3. Desplácese a la sección de **Binary Distributions** y descargue el paquete comprimido denominado **cmake-4.3.2-windows-x86_64.zip** adaptado para la arquitectura de Windows de 64 bits.

Uso de la Versión Portátil (ZIP)

Al tratarse de una versión comprimida en formato .zip, deberá extraer el contenido en una carpeta permanente dentro de su disco duro (por ejemplo, en C: ▶ Program Files ▶ CMake o C: ▶ CMake). No olvide agregar la ruta de la subcarpeta bin a las variables de entorno (PATH) del sistema para que Visual Studio Code pueda ejecutar CMake de forma global.

2 CONFIGURACIONES PREVIAS EN VISUAL STUDIO CODE

Antes de proceder con la inicialización y creación de cualquier proyecto, es muy importante realizar un par de ajustes de preparación dentro de Visual Studio Code. Estas configuraciones se realizan una sola vez en el editor, por lo que no será necesario repetirlas cada vez que cree un nuevo proyecto en el futuro.

2.1 Creación y Uso de Perfiles (Profiles)

Los **Perfiles (Profiles)** en Visual Studio Code son una característica que permite guardar y aislar conjuntos específicos de configuraciones, temas, atajos de teclado, estados de la interfaz y extensiones instaladas.

2.1.1 ¿Para qué sirven y cuáles son sus ventajas?

El desarrollo embebido para microcontroladores STM32 requiere herramientas y dependencias muy particulares (compiladores, extensiones de depuración, etc.). El uso de un perfil dedicado aporta las siguientes ventajas:

- **Aislamiento y Organización:** Mantiene las extensiones de STM32 separadas de las extensiones que utilice para otros tipos de programación (como desarrollo web, Python, etc.), evitando conflictos y saturación de menús.
- **Optimización del Rendimiento:** Al cargar únicamente las extensiones específicas de STM32 en este perfil, VS Code consume menos memoria RAM y el editor arranca mucho más rápido.
- **Portabilidad y Respaldo:** Permite exportar toda la configuración de desarrollo en un solo archivo e importarlo en cualquier otra máquina para tener el entorno de trabajo listo al instante.

2.1.2 Pasos para crear un nuevo perfil

Para configurar un perfil exclusivo para su trabajo con tarjetas STM, siga los siguientes pasos:

1. Diríjase al extremo inferior de la barra de actividad lateral izquierda y haga clic en el icono del **engranaje** (Administración).
2. En el menú contextual que aparece, seleccione la opción **Profiles** → **Profiles** (o **Create Profile...** según la versión de su editor).
3. Esto abrirá una pestaña de configuración llamada **"Profiles"**. En el campo de texto etiquetado como **Name**, asigne un nombre representativo para este perfil, por ejemplo: STM32 (tal como se ilustra en la figura 1).

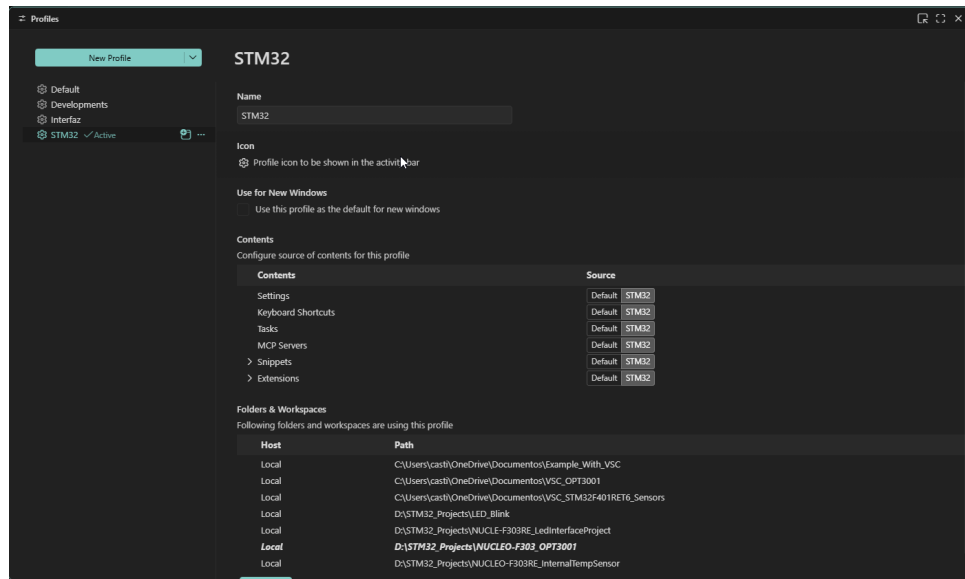


Figura 1: Creación del perfil dedicado "STM32" en Visual Studio Code.

- Una vez ingresado el nombre, puede proceder a cerrar la pestaña de perfiles; el entorno guardará los cambios automáticamente.

2.1.3 Activación del perfil

Para activar y comenzar a trabajar bajo este entorno, haga clic nuevamente en el icono del **engranaje** en la esquina inferior izquierda, desplace el cursor sobre la opción **Profiles** y haga clic en el perfil previamente creado (**STM32**) en la lista desplegable. Esto cambiará todo el contexto del editor a tu espacio de trabajo configurado.

2.2 Configuración de la Ruta del Ejecutable de CMake

De acuerdo con la documentación y advertencias provistas por la extensión oficial **CMake Tools**, para que la automatización de compilación opere sin inconvenientes es necesario cumplir con el siguiente requerimiento:

"Ensure that either you've added your CMake executable to your PATH, or you've adjusted the cmake.cmakePath setting to point to your CMake executable."

Para solventar esta directiva indicando explícitamente a VS Code dónde se encuentra el ejecutable de CMake en su computadora, realice el siguiente procedimiento:

- Diríjase al icono del **engranaje** (Administración) en la barra de actividad y elija la opción **Settings** (o use el atajo rápido **Ctrl** + **,**).
- En la barra de búsqueda de la pestaña de configuraciones que se despliega, escriba la frase de búsqueda: `path cmake`.
- Identifique el campo de configuración titulado **"Cmake: Cmake Path"** (cuya etiqueta de descripción indica: *Name/path of the CMake executable to use*).
- En la caja de texto de esta opción, introduzca la ruta exacta en su disco duro donde se localiza el binario de CMake. Para esta instalación, la ruta corresponde a: `C:\Program Files\CMake\bin\cmake` (tal como se ilustra en la figura 2).

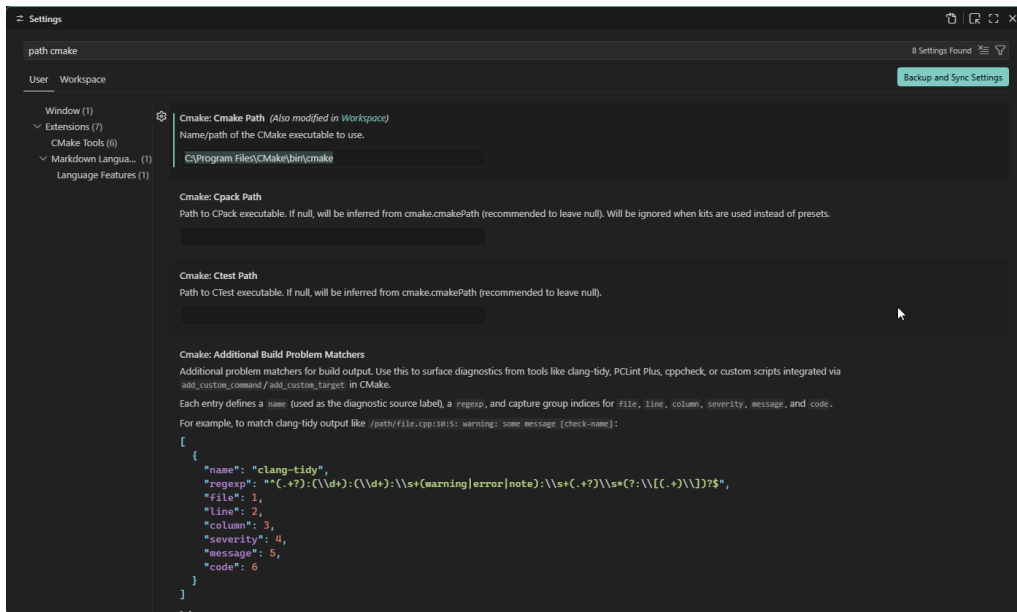


Figura 2: Asignación de la ruta de CMake en el campo Cmake Path de VS Code.

5. Cierre la ventana de configuración; Visual Studio Code registrará y salvará las modificaciones de forma automática e inmediata.

2.3 Configuración de la Ruta del Ejecutable de STM32CubeMX

Para garantizar que Visual Studio Code pueda lanzar la herramienta gráfica de configuración física de pines sin inconvenientes, es altamente recomendable configurar de forma explícita la ruta de instalación de STM32CubeMX. Esto previene un error común en el que el editor muestra una notificación indicando que no se ha encontrado la ruta para abrir la aplicación. Para configurar esta ruta, realice el siguiente procedimiento:

1. Abra la ventana de configuraciones globales de Visual Studio Code mediante el atajo de teclado **Ctrl** + **,** (o dirigiéndose al icono del engranaje en la esquina inferior izquierda y seleccionando **Settings**).
2. En la barra de búsqueda de la pestaña de configuraciones, escriba el texto de búsqueda: `path stm32cubemx`.
3. Identifique la sección de configuración correspondiente a la extensión de STMicroelectronics, específicamente el campo titulado:
Stm32cube-ide-core → **Configuration** → **Product STM32 Cube MX: Executable Path**
(Absolute path to the STM32CubeMX executable).
4. Introduzca la ruta absoluta exacta donde se encuentra instalado el ejecutable en su sistema. Para esta versión, la ruta corresponde a:
`C:\Users\casti\AppData\Local\Programs\STM32CubeMX_6.17.0\STM32CubeMX.exe`
 (tal como se ilustra en la figura 3).

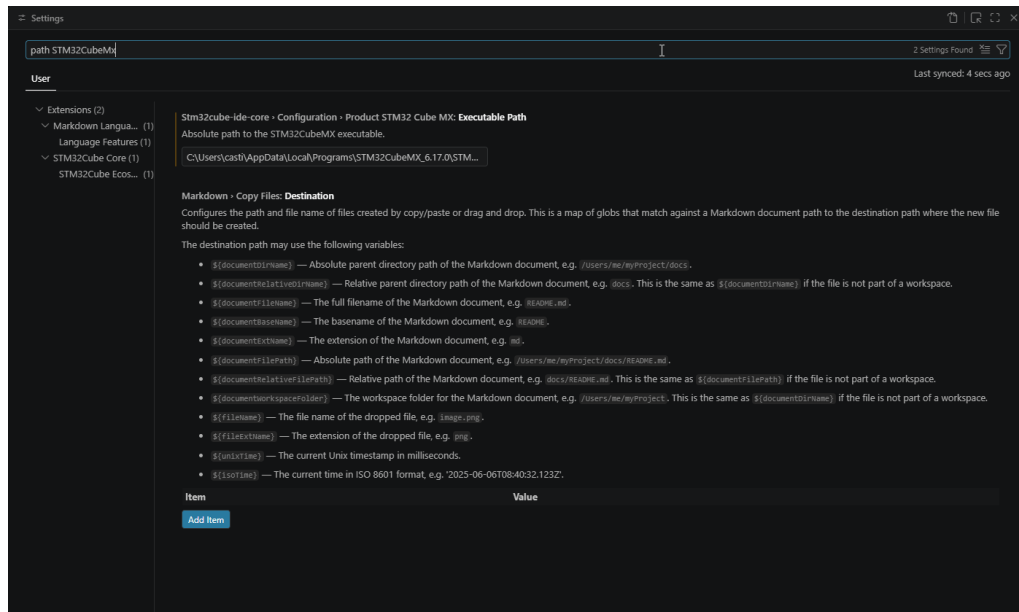


Figura 3: Asignación de la ruta absoluta del ejecutable de STM32CubeMX en VS Code.

5. Cierre la ventana de configuración; Visual Studio Code registrará y salvará las modificaciones de forma automática e inmediata.

⚠ Error de Lanzamiento de STM32CubeMX

Si omite esta configuración, al hacer clic en las opciones del panel lateral de ST como **Launch STM32CubeMX**, es posible que VS Code intente buscar el ejecutable en las rutas por defecto del sistema y falle si se trata de una instalación local de usuario, arrojando una alerta de error. Configurar la ruta absoluta soluciona este comportamiento de manera permanente.

3 CREACIÓN DE UN NUEVO PROYECTO USANDO STM32CUBEMX

Una vez que el entorno de desarrollo y las extensiones se han instalado correctamente, puede proceder con la creación y configuración inicial de su firmware. El flujo de trabajo oficial de ST en Visual Studio Code le permite gestionar todo el proceso desde el panel integrado:

1. Diríjase a la **barra de actividad lateral izquierda** en Visual Studio Code.
2. Haga clic en el icono oficial de **STM32Cube** (el logotipo con las letras **ST**).
3. Se abrirá un panel lateral en el editor que mostrará las siguientes 7 opciones de configuración y creación (figura 4):
 - **Launch STM32CubeMX:** Abre de forma externa la herramienta gráfica oficial de ST para configurar el hardware y los periféricos del microcontrolador.
 - **Launch STM32CubeMX2:** Método secundario de acceso rápido para arrancar la interfaz gráfica de CubeMX.
 - **Launch STM32CubeFinder:** Lanza el catálogo interactivo de STMicroelectronics para buscar y elegir chips o tarjetas de desarrollo específicas.
 - **Create Empty Project:** Inicializa la estructura básica de carpetas y código para un proyecto de STM32 desde cero.
 - **Import STM32Cube example(s):** Permite explorar e importar proyectos de ejemplo predefinidos provistos por ST para diferentes periféricos.
 - **Convert eclipse STM32CubeIDE project:** Utilidad para migrar proyectos antiguos que fueron configurados bajo el IDE tradicional basado en Eclipse hacia el nuevo formato de Visual Studio Code.
 - **Set Up STM32Cube project(s):** Parámetros avanzados de configuración para proyectos STM32 (*esta opción se encuentra deshabilitada por el momento*).

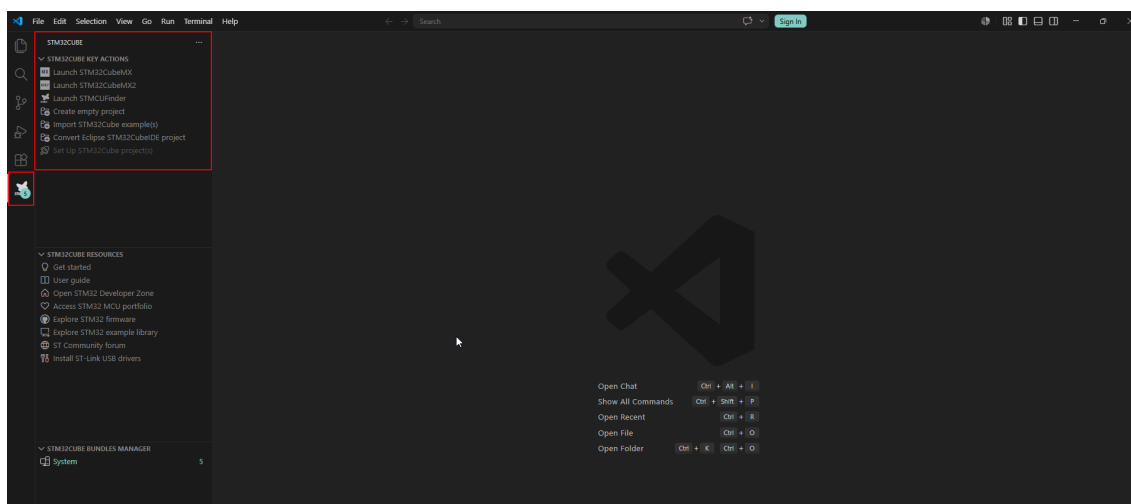


Figura 4: STM32Cube - opciones de trabajo

Para continuar con el proceso de creación y configuración física de la tarjeta de desarrollo, siga los pasos a continuación:

4. Seleccione la opción **Launch STM32CubeMX** en el menú lateral. Esto abrirá la interfaz de configuración gráfica de STM32CubeMX en una ventana independiente, tal como se muestra en la figura 5.

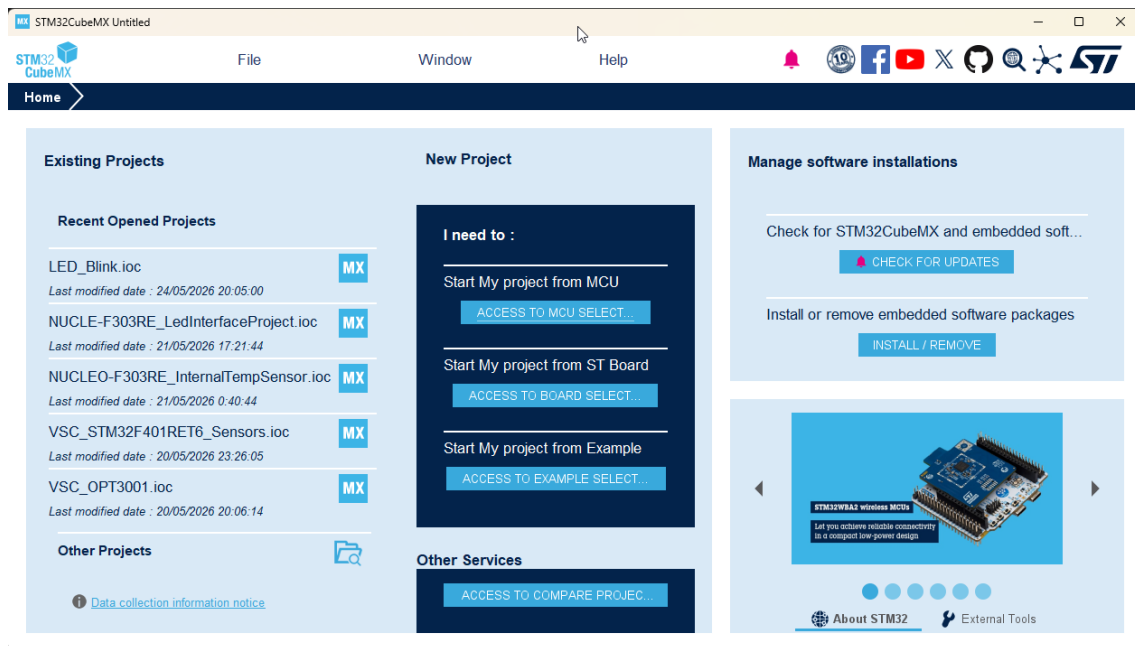


Figura 5: Interfaz de bienvenida de STM32CubeMX.

5. Dado que para este proyecto se utiliza la tarjeta de desarrollo **NUCLEO-F303RE**, busque en la pantalla central de STM32CubeMX la sección **Start my project from ST Board** y haga clic en el botón **ACCESS TO BOARD SELECT...**
6. Esta acción abrirá la ventana de selección de hardware titulada "New Project from a Board" (ilustrada en la figura 6), la cual carga por defecto en la pestaña **Board Selector**.

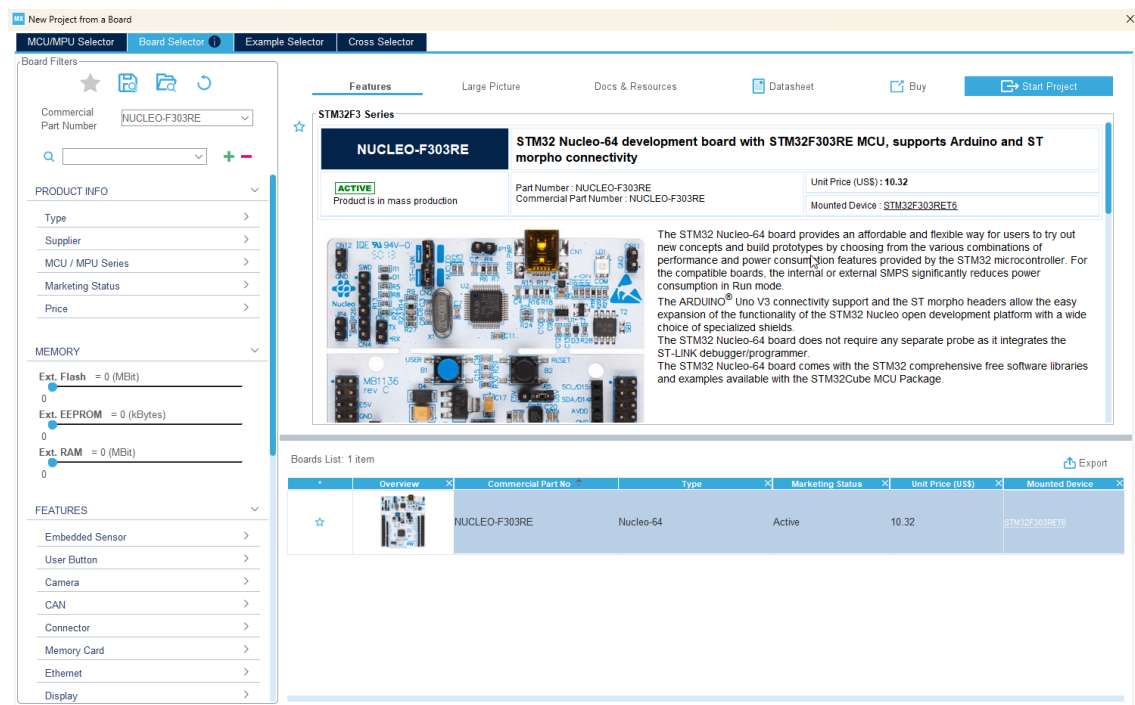


Figura 6: Ventana "New Project from a Board" con filtro de búsqueda.

7. En el menú de filtros del lado izquierdo de esta ventana, busque el campo de texto etiquetado como **Commercial Part Number**. Escriba allí el modelo de la placa: **NUCLEO-F303RE**.

8. Al mostrarse la tarjeta en la sección central de resultados, selecciónela haciendo clic sobre su imagen/descripción.
9. Finalmente, haga clic en el botón **Start Project** (ubicado en la esquina superior derecha) para inicializar el proyecto base con la arquitectura de hardware de la NUCLEO-F303RE.

Para la configuración de periféricos, configuración del reloj y exportación del proyecto, prosiga con los siguientes pasos:

10. Al iniciar el proyecto se abrirá el entorno principal de edición de STM32CubeMX, donde podrá personalizar las características del hardware del microcontrolador:
 - **Pinout & Configuration:** Sección para configurar los pines de entrada/salida (GPIO) y activar los recursos de hardware necesarios como Timers, ADC/DAC e Interfaces de comunicación física (I2C, SPI, UART/USART, etc.).
 - **Clock Configuration:** Pestaña visual interactiva para ajustar la frecuencia de reloj del procesador, configurar multiplicadores/divisores PLL y seleccionar las fuentes de oscilador.
11. Una vez completada la configuración de hardware de su microcontrolador, diríjase a la pestaña superior titulada **Project Manager**.
12. En el menú lateral izquierdo, seleccione la sección **Project** y llene los campos requeridos en la ventana **Project Settings** (de la misma manera que se muestra en la figura 7):

PROJECT NAME: El nombre que tendrá la carpeta y los archivos del firmware (por ejemplo: NUCLEO-F303_OPT3001).

PROJECT LOCATION: La carpeta raíz de su computadora donde desea almacenar el proyecto (por ejemplo: D:\STM32_Projects\).

TOOLCHAIN FOLDER LOCATION: Esta ruta se autocompleta automáticamente concatenando los campos anteriores (por ejemplo: D:\STM32_Projects\NUCLEO-F303_OPT3001\).

TOOLCHAIN / IDE: En la caja de selección desplegable, elija específicamente la opción **CMake**.

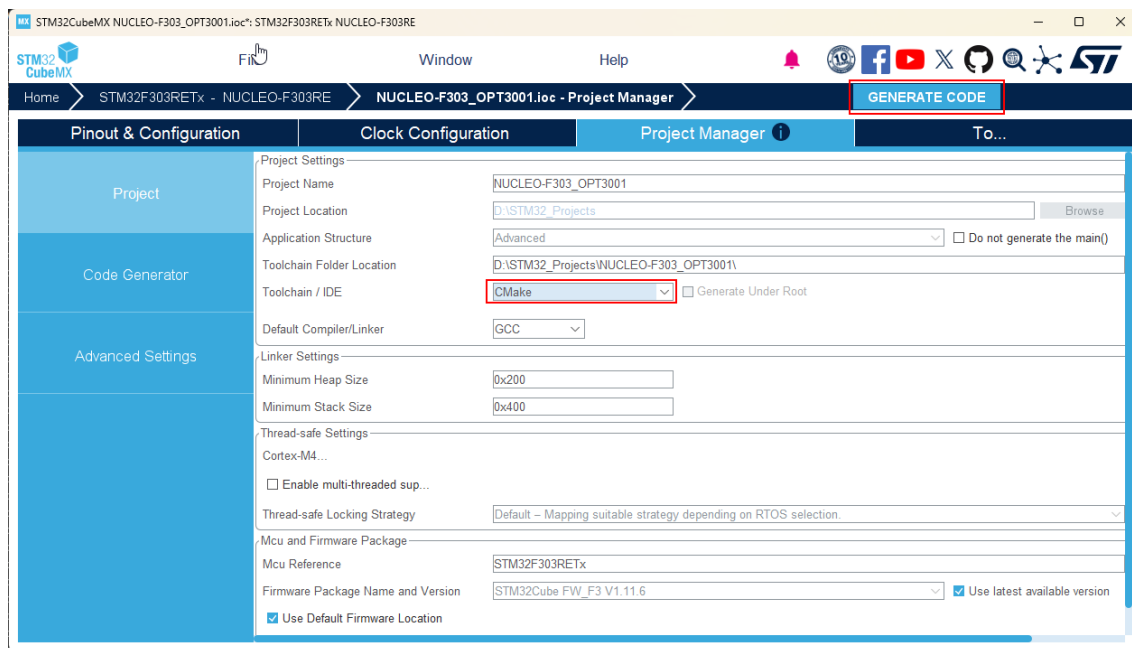


Figura 7: Parámetros de configuración del proyecto en la pestaña Project Manager.

13. Una vez que haya completado todos los pasos de configuración de esta pestaña, presione el botón **GENERATE CODE** localizado en la barra de menús superior derecha. Esto generará

la estructura de carpetas de código fuente C/C++ y los archivos de compilación para CMake.

14. Regrese a Visual Studio Code. Abra el panel lateral del Explorador de archivos con el atajo **Ctrl**+**Shift**+**E** (o a través de la barra de menús en **File** → **Open Folder**) y abra la carpeta del proyecto recién creado (**D:** ▶ **STM32_Projects** ▶ **NUCLEO-F303_OPT3001** ▶).
15. Una vez cargada la carpeta en VS Code, deberá visualizar el árbol de directorios y archivos generado por la herramienta (tal como se ilustra en la figura 8), compuesto principalmente por:
 - **cmake** ▶ (Directorio con los archivos de la Toolchain de GCC)
 - **Core** ▶ (Carpeta que contiene el código fuente principal, organizada en **Src** e **Inc**)
 - **Drivers** ▶ (Controladores HAL y CMSIS oficiales de STMicroelectronics)
 - **.mxproject** (Archivo de configuración interna para CubeMX)
 - **CMakeLists.txt** (Archivo de configuración de compilación de CMake)
 - **CMakePresets.json** (Definición de presets de desarrollo y pruebas de CMake)
 - Archivo con extensión **.ioc** (Fichero de diseño gráfico del microcontrolador de CubeMX)
 - **startup_stm32f303xe.s** (Código assembler para el vector de interrupciones y arranque del chip)
 - **STM32F303XX_FLASH.id** (Archivo del mapa de memoria / Linker Script del enlazador)

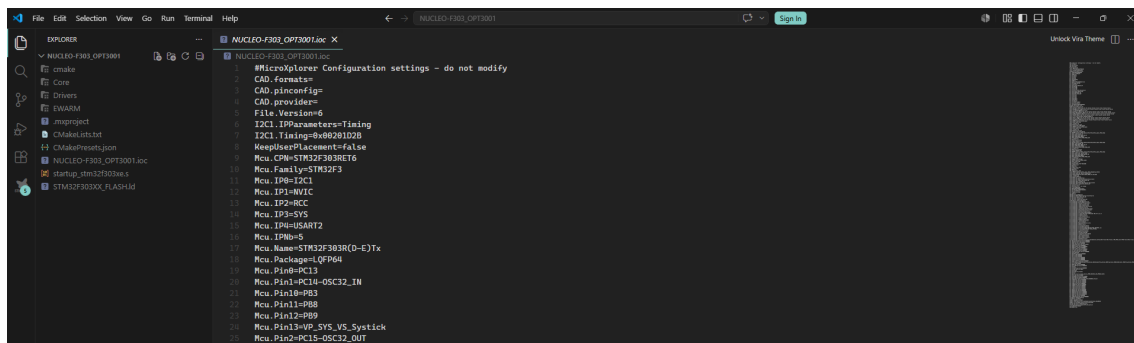


Figura 8: Estructura del proyecto recién creado abierta en el explorador de VS Code.

16. Diríjase a la barra de actividad lateral izquierda en VS Code y seleccione de nuevo el icono de **STM32Cube** (logotipo de ST).
17. Con su tarjeta de desarrollo **NUCLEO-F303RE** conectada físicamente por USB a su computadora, haga clic en la opción del menú lateral titulada **Set Up STM32Cube project(s)**.
18. Esta acción abrirá la pestaña interactiva de configuración de hardware titulada **"Project Setup"** en VS Code. Aquí deberá seleccionar las siguientes opciones:
 - **Board/Device:** Seleccione la tarjeta del proyecto (en este ejemplo: **NUCLEO-F303RE**).
 - **Toolchain:** Seleccione la opción de compilador **GCC**.
19. Una vez seleccionadas estas dos opciones, presione el botón **Configure**.
20. A continuación, se desplegará un menú en la barra de búsqueda superior de VS Code para que seleccione el preset de compilación del proyecto. Seleccione la opción **Debug** para este caso de estudio (como se muestra en la figura 9).

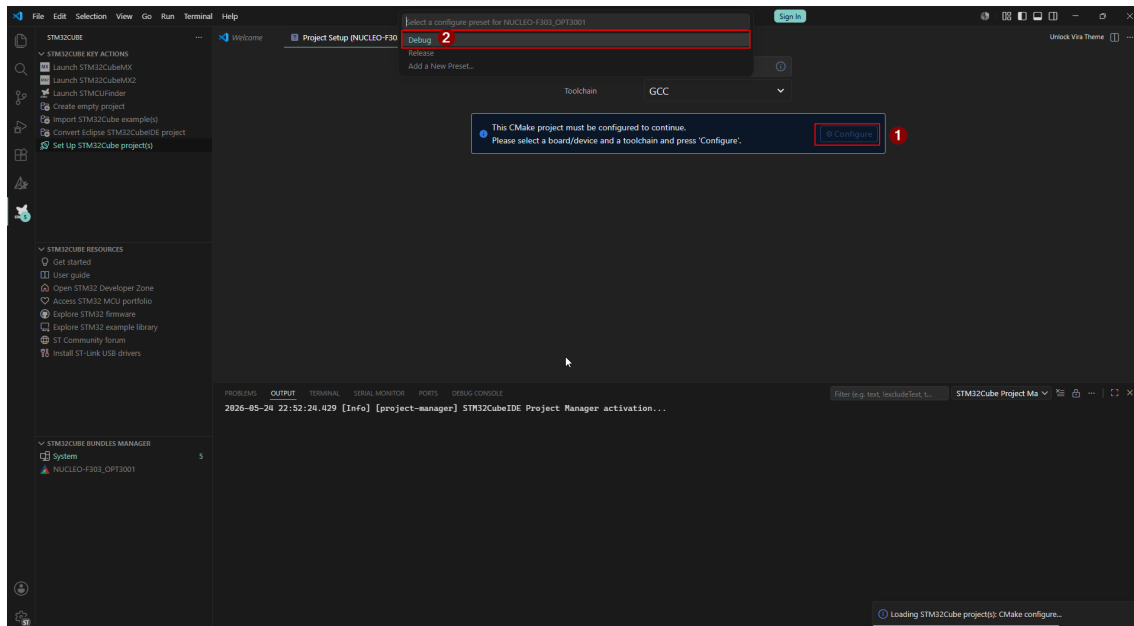


Figura 9: Selección del preset Debug en la barra de entrada de VS Code.

Al seleccionar esta opción, el motor de CMake comenzará a configurar el proyecto y verá la siguiente salida en la terminal integrada:

```
[bookmarks] Loaded 0 bookmarks
[proc] Executing command: cube-cmake --version
[proc] Executing command: cube-cmake -E capabilities
[kit] Successfully loaded 4 kits from C:\Users\casti\AppData\Local\CMakeTools\cmake-tools-kits.json
[presetController] Reading and validating the presets file "D:\STM32_Projects\NUCLE0-F303_OPT3001\
CMakePresets.json"
[presetController] Successfully validated D:\STM32_Projects\NUCLE0-F303_OPT3001\CMakePresets.json
against presets schema
[presetController] Expanding presets file D:\STM32_Projects\NUCLE0-F303_OPT3001\CMakePresets.json
[presetController] Successfully expanded presets file D:\STM32_Projects\NUCLE0-F303_OPT3001\CMakePresets
.json
[driver] Switching to configure preset: Debug
[driver] Switching to build preset: Debug
[driver] Switching to test preset: __defaultTestPreset__
[driver] Switching to test preset: __defaultTestPreset__
[driver] Switching to package preset: __defaultPackagePreset__
[driver] Switching to workflow preset: __defaultWorkflowPreset__
[main] Configuring project: NUCLE0-F303_OPT3001
[driver] NOTE: You are configuring with preset Debug, but there are some overrides being applied from
your VS Code settings.
[proc] Executing command: cube-cmake -DCMAKE_BUILD_TYPE=Debug -DCMAKE_TOOLCHAIN_FILE=D:/STM32_Projects/
NUCLE0-F303_OPT3001/cmake/gcc-arm-none-eabi.cmake -DCMAKE_COMMAND=cube-cmake -
DCMAKE_EXPORT_COMPILE_COMMANDS=BOOL=TRUE -S D:/STM32_Projects/NUCLE0-F303_OPT3001 -B D:/
STM32_Projects/NUCLE0-F303_OPT3001/build/Debug -G Ninja
[cmake] Build type: Debug
[cmake] -- Found assembler: C:/Users/casti/AppData/Local/stm32cube/bundles/gnu-tools-for-stm32/14.3.1+st
.2/bin/arm-none-eabi-gcc.exe
[cmake] -- Configuring done (5.4s)
[cmake] -- Generating done (0.1s)
[cmake] CMake Warning:
[cmake]   Manually-specified variables were not used by the project:
[cmake]
[cmake]     CMAKE_COMMAND
[cmake]
[cmake] -- Build files have been written to: D:/STM32_Projects/NUCLE0-F303_OPT3001/build/Debug
```

21. Para finalizar y almacenar esta configuración, haga clic en el botón **Save and Close** en el panel superior de configuración (ver figura 10).

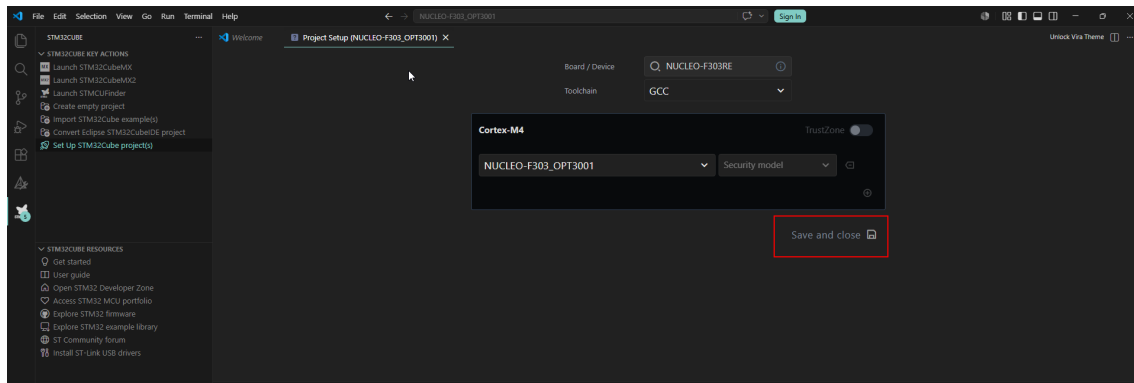


Figura 10: Guardar la configuración final en la ventana de Project Setup.

Esto desencadenará la recarga de CMake y mostrará el siguiente reporte final en su terminal confirmando la correcta escritura de los archivos de compilación (*build*):

```
[main] Configuring project: NUCLEO-F303_OPT3001
[driver] Removing D:/STM32_Projects/NUCLEO-F303_OPT3001/build/Debug/CMakeCache.txt
[driver] Removing D:\%5CSTM32_Projects\%5CNUCLEO-F303_OPT3001\%5Cbuild\%5CDebug\%5CCMakeFiles
[driver] NOTE: You are configuring with preset Debug, but there are some overrides being applied from
your VS Code settings.
[proc] Executing command: cube-cmake -DCMAKE_BUILD_TYPE=Debug -DCMAKE_TOOLCHAIN_FILE=D:/STM32_Projects/
NUCLEO-F303_OPT3001/cmake/gcc-arm-none-eabi.cmake -DCMAKE_COMMAND=cube-cmake -
DCMAKE_EXPORT_COMPILE_COMMANDS=BOOL=TRUE -S D:/STM32_Projects/NUCLEO-F303_OPT3001 -B D:/
STM32_Projects/NUCLEO-F303_OPT3001/build/Debug -G Ninja
[cmake] Build type: Debug
[cmake] -- Found assembler: C:/Users/casti/AppData/Local/stm32cube/bundles/gnu-tools-for-stm32/14.3.1+st
.2/bin/arm-none-eabi-gcc.exe
[cmake] -- Configuring done (1.8s)
[cmake] -- Generating done (0.1s)
[cmake] CMake Warning:
[cmake]   Manually-specified variables were not used by the project:
[cmake]
[cmake]     CMAKE_COMMAND
[cmake]
[cmake] -- Build files have been written to: D:/STM32_Projects/NUCLEO-F303_OPT3001/build/Debug
```

i ¿No aparece el selector de Preset (Debug / Release)?

Si después de hacer clic en **Configure** la barra superior de comandos de VS Code no muestra de forma automática el menú desplegable para elegir el preset de compilación, puede forzar su aparición de la siguiente manera:

1. Diríjase a la barra de actividad lateral izquierda y seleccione la pestaña de **CMake** (el icono de la figura geométrica).
2. En el panel que se despliega, busque la sección titulada **Configure**.
3. Pase el cursor sobre dicha sección y haga clic en el **icono de lápiz (Editar)**.
4. Esto forzará el despliegue del menú de selección en la parte superior de la pantalla para que pueda elegir el preset deseado (como se ilustra en la figura 11).

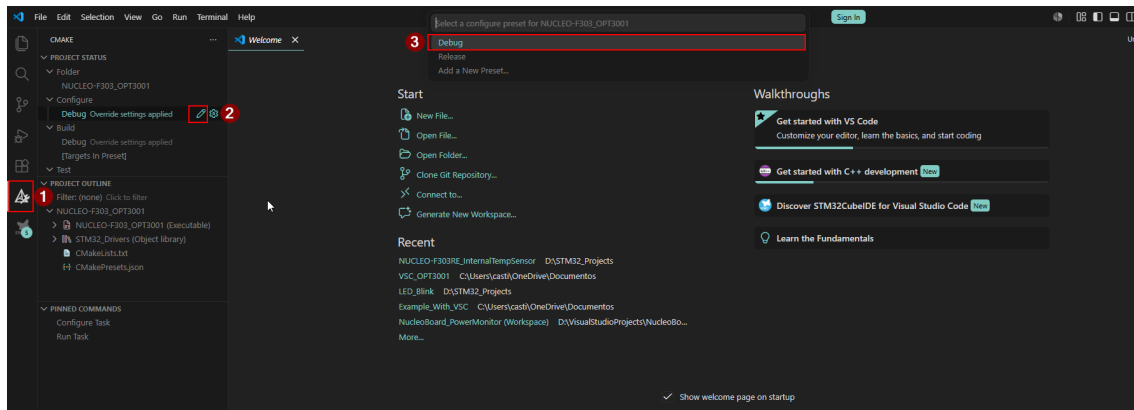


Figura 11: Selección forzada de presets desde el panel de control de CMake.

3.1 Prueba de Funcionamiento: Parpadeo de LED (Blinky)

Para verificar que el flujo de trabajo completo (compilador, enlazador, CMake y el servidor de depuración ST-Link) funciona adecuadamente, realizaremos una prueba clásica haciendo parpadear el LED integrado en la propia placa.

20. En el Explorador de archivos de VS Code, diríjase a la ruta `Core>Src>main.c` y abra dicho archivo de código fuente C.
21. Busque el ciclo infinito principal `while` (1) dentro de la función `main` y coloque las siguientes líneas para alternar el estado del pin del LED (el cual para la placa NUCLEO-F303RE está asociado físicamente a la etiqueta `GPIOA` en el `GPIO_PIN_5`):

```

1      /* USER CODE BEGIN 3 */
2      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
3      HAL_Delay(100);
4      /* USER CODE END 3 */
5

```

i Uso de las etiquetas **USER CODE**

Es sumamente importante colocar el código que escriba únicamente entre los bloques marcados con comentarios de tipo `/* USER CODE BEGIN */` y `/* USER CODE END */`. De lo contrario, si realiza cambios futuros en los periféricos desde STM32CubeMX y regenera el código, las líneas externas a estas marcas se borrarán de forma definitiva.

22. Guarde el archivo e inicie la compilación del proyecto seleccionando la opción **Build** localizada en la barra de estado inferior izquierda de la ventana de VS Code (tal como se ilustra en la figura 12).

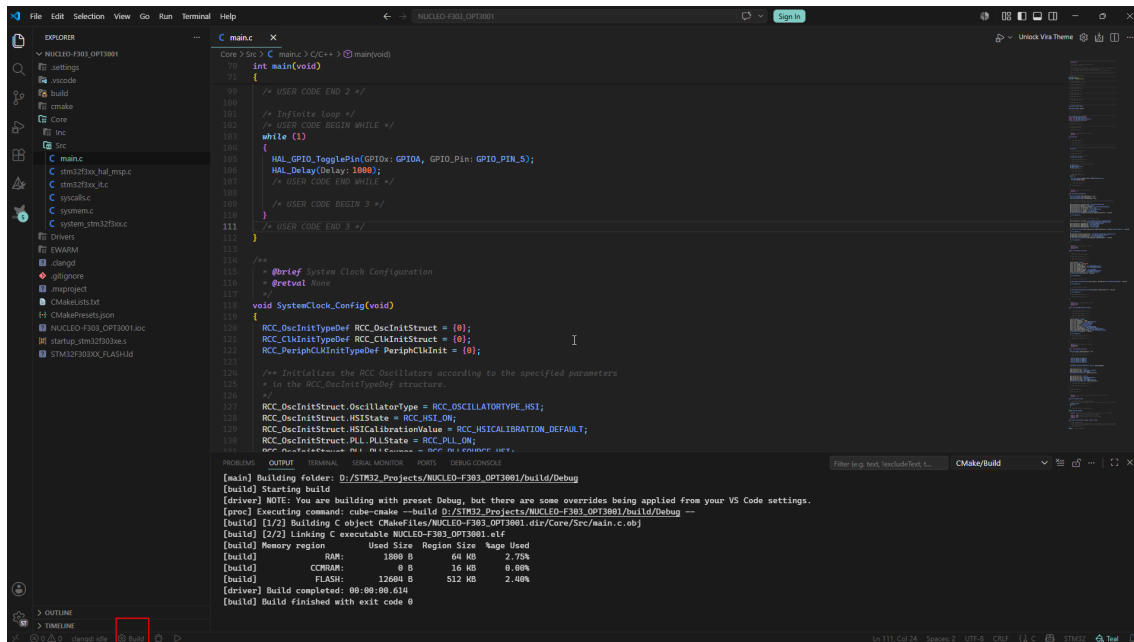


Figura 12: Ubicación de la opción Build para compilar el proyecto en la barra inferior.

La terminal de VS Code desplegará el avance de compilación indicando las regiones de memoria ocupadas por el binario resultante:

```
[main] Building folder: D:/STM32_Projects/NUCLEO-F303_OPT3001/build/Debug
[build] Starting build
[driver] NOTE: You are building with preset Debug, but there are some overrides being applied from your VS Code settings.
[proc] Executing command: cube-cmake --build D:/STM32_Projects/NUCLEO-F303_OPT3001/build/Debug
--
[build] [1/2] Building C object CMakeFiles/NUCLEO-F303_OPT3001.dir/Core/Src/main.c.obj
[build] [2/2] Linking C executable NUCLEO-F303_OPT3001.elf
[build] Memory region      Used Size  Region Size  %age Used
[build]      RAM:          1800 B       64 KB      2.75%
[build]      CCMRAM:         0 B        16 KB      0.00%
[build]      FLASH:       12604 B       512 KB      2.40%
[driver] Build completed: 00:00:00.614
[build] Build finished with exit code 0
```

23. Una vez compilado con éxito el proyecto, haga clic en el icono de **Run and Debug** (icono del insecto con botón play) en la barra de actividad o presione el atajo **Ctrl** + **Shift** + **D**.
24. En el menú desplegable de arranque de la parte superior del panel, se mostrarán las siguientes configuraciones de lanzamiento disponibles:
 - C++ (GDB/LLDB)
 - C++ (Windows)
 - STM32Cube: STM32 launch GDB
 - STM32Cube: STM32 launch JLink GDB Server
 - STM32Cube: STM32 launch STLink GDB Server
25. Seleccione exactamente la opción **STM32Cube: STM32 launch STLink GDB Server** (ver figura 13) y haga clic en el botón verde de Play para iniciar la depuración del microcontrolador.

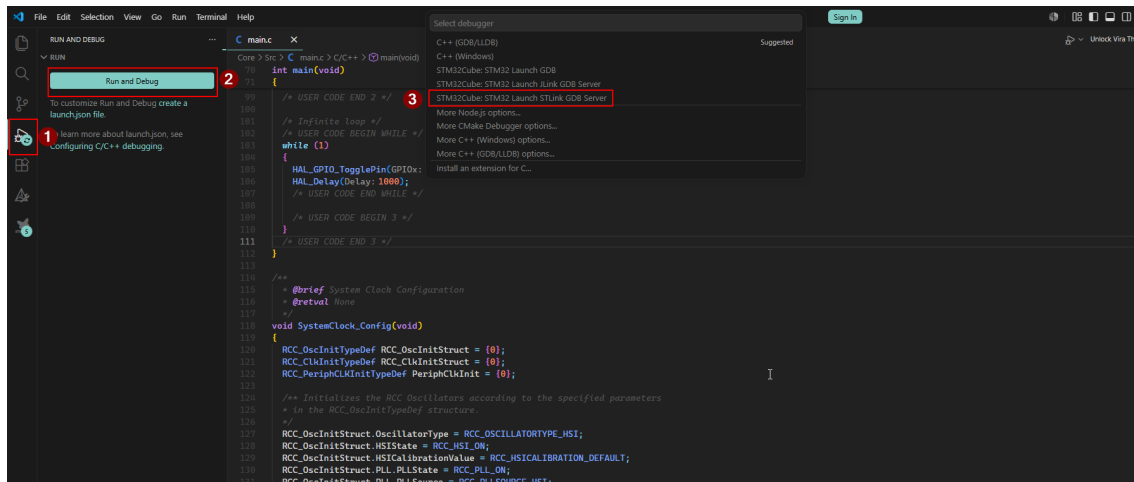


Figura 13: Selección de la configuración de depuración para usar STLink GDB Server.

Generación del archivo JSON

Antes de ejecutar el debug, tiene que generar el archivo **launch.json**, para esto, seleccione la opción: *To customize Run and Debug create a launch.json file*, ubicado debajo del botón **Run and Debug**.

26. Esto establecerá la comunicación con su tarjeta y abrirá la pestaña **DEBUG CONSOLE** (Consola de depuración), mostrando los mensajes de conexión del programador y volcado de memoria a la Flash:

```
STMicroelectronics ST-LINK GDB server. Version 7.13.0
Copyright (c) 2026, STMicroelectronics. All rights reserved.

Starting server with the following options:
Persistent Mode      : Disabled
Logging Level       : 1
Listen Port Number   : 61234
Status Refresh Delay : 15s
Verbose Mode        : Disabled
SWD Debug           : Enabled
InitWhile           : Enabled

Waiting for debugger connection...
GNU gdb (GNU Tools for STM32 14.3.rel1.20251027-0700) 15.2.90.20241229-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-w64-mingw32 --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
add-symbol-file "D:/STM32_Projects/NUCLE0-F303_OPT3001/build/Debug/NUCLE0-F303_OPT3001.elf"
add symbol table from file "D:/STM32_Projects/NUCLE0-F303_OPT3001/build/Debug/NUCLE0-
F303_OPT3001.elf"
(y or n) [answered Y; input not from terminal]
Reading symbols from D:/STM32_Projects/NUCLE0-F303_OPT3001/build/Debug/NUCLE0-F303_OPT3001.elf
...
Debugger connected
Waiting for debugger connection...
0x0800167c in __RBIT (value=134223565) at D:/STM32_Projects/NUCLE0-F303_OPT3001/cmake/
stm32cubemx/../../Drivers/CMSIS/Include/cmsis-gcc.h:988
988      __ASM volatile ("rbit %0, %1" : "=r" (result) : "r" (value) );
connected to remote target localhost:61234
tbreak main
```

```

Temporary breakpoint 1 at 0x8002bd6: file D:/STM32_Projects/NUCLE0-F303-OPT3001/Core/Src/main.c,
line 80.
Note: automatically using hardware breakpoints for read-only addresses.
load "D:/STM32_Projects/NUCLE0-F303-OPT3001/build/Debug/NUCLE0-F303-OPT3001.elf"
Loading section .isr_vector, size 0x194 lma 0x8000000
Loading section .text, size 0x2f5c lma 0x8000194
Loading section .rodata, size 0x38 lma 0x80030f0
Loading section .init_array, size 0x4 lma 0x8003128
Loading section .fini_array, size 0x4 lma 0x800312c
Loading section .data, size 0xc lma 0x8003130
-----
STM32CubeProgrammer v2.22.0
-----
3

Log output file: C:\Users\casti\AppData\Local\Temp\STM32CubeProgrammer_a05412.log
ST-LINK SN : 066AFF484971754867123220
ST-LINK FW : V2J46M31
Board : NUCLE0-F303RE
Voltage : 3.25V
SWD freq : 4000 KHz
Connect mode: Under Reset
Reset mode : Hardware reset
Device ID : 0x446
Revision ID : Rev Y
Device name : STM32F302xE/F303xE/F398xx
NVM size : 512 KBytes
Device type : MCU
Device CPU : Cortex-M4
BL Version : 0x40
3

Opening and parsing file: ST-LINK_GDB_server_a05412.srec
2

Memory Programming ...
File : ST-LINK_GDB_server_a05412.srec
Size : 12.31 KB
Address : 0x08000000
2

Erasing memory corresponding to segment 0:
Erasing internal memory sectors [0 6]
Download in Progress:
2

File download complete
Time elapsed during download operation: 00:00:00.693
Start address 0x0800302c, load size 12604
Transfer rate: 6 KB/sec, 2100 bytes/write.
set *(int *)0xE000ED14=*(int *)0xE000ED14|0x10
set *(int *)0xE000EDFC=*(int *)0xE000EDFC|0x7f0
set *(unsigned int *)0xe0042004|=0x7
set *(unsigned int *)0xe0042008|=0x1800

In the Debug Console view you can interact directly with GDB.
To display the value of an expression, type that expression which can reference
variables that are in scope. For example type '2 + 3' or the name of a variable.
Arbitrary commands can be sent to GDB by prefixing the input with a '>',
for example type '>show version' or '>help'.

Temporary breakpoint 1, main () at D:/STM32_Projects/NUCLE0-F303-OPT3001/Core/Src/main.c:80
80 HAL_Init();

```

27. Al finalizar la carga, observará que la aplicación se pausa en un punto de interrupción temporal, deteniendo la ejecución al inicio del código en la función `HAL_Init()`. Utilice la barra de herramientas flotante de depuración de la parte superior para reanudar o controlar el flujo paso a paso (ver figura 14).

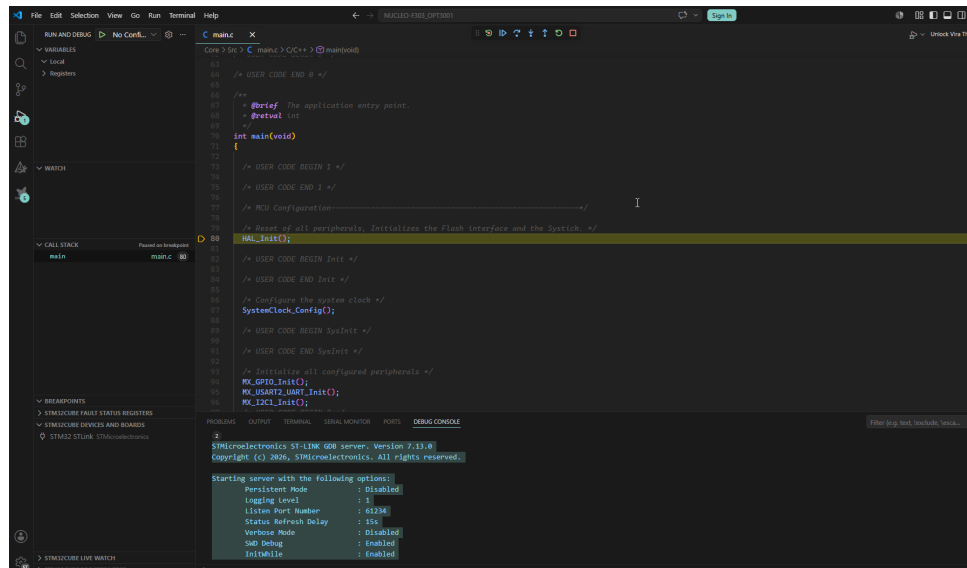


Figura 14: Barra flotante superior de control del depurador en VS Code.

28. Presione el botón de **Continue** (o pulse la tecla **F5**) para liberar la pausa de depuración. Inmediatamente después de esto, el firmware continuará su ciclo normal y verá cómo el LED verde integrado de su tarjeta física comienza a parpadear secuencialmente cada 100 ms.

4 INTEGRACIÓN DE CONTROLADORES Y LIBRERÍAS PROPIAS EN CMAKE

Al trabajar bajo el entorno de compilación CMake en Visual Studio Code, el compilador no incluye automáticamente carpetas o archivos que se agreguen de manera manual al proyecto. Si se añade una carpeta de biblioteca (por ejemplo, el controlador del sensor OPT3001 en la ruta Core▶OPT3001 conteniendo las subcarpetas Inc y Src), se deben realizar modificaciones en el archivo CMakeLists.txt de la raíz para evitar errores de compilación por archivos faltantes.

4.1 Estructura del archivo CMakeLists.txt (Antes)

Originalmente, el archivo de configuración de CMake generado por STM32CubeMX define la estructura básica del proyecto, pero deja los campos de fuentes e inclusiones del usuario vacíos:

```

1  cmake_minimum_required(VERSION 3.22)
2  #
3  # This file is generated only once,
4  # and is not re-generated if converter is called multiple times.
5  #
6  # User is free to modify the file as much as necessary
7  #
8  # Setup compiler settings
9  set(CMAKE_C_STANDARD 11)
10 set(CMAKE_C_STANDARD_REQUIRED ON)
11 set(CMAKE_C_EXTENSIONS ON)
12 # Define the build type
13 if(NOT CMAKE_BUILD_TYPE)
14   set(CMAKE_BUILD_TYPE "Debug")
15 endif()
16 # Set the project name
17 set(CMAKE_PROJECT_NAME NUCLE0-F303-OPT3001)
18 # Enable compile command to ease indexing with e.g. clangd
19 set(CMAKE_EXPORT_COMPILE_COMMANDS TRUE)
20 # Core project settings
21 project(${CMAKE_PROJECT_NAME})
22 message("Build type: " ${CMAKE_BUILD_TYPE})
23 # Enable CMake support for ASM and C languages
24 enable_language(C ASM)
25 # Create an executable object type
26 add_executable(${CMAKE_PROJECT_NAME})
27 # Add STM32CubeMX generated sources
28 add_subdirectory(cmake/stm32cubemx)
29 # Link directories setup
30 target_link_directories(${CMAKE_PROJECT_NAME} PRIVATE
31 # Add user defined library search paths
32 )
33 # Add sources to executable
34 target_sources(${CMAKE_PROJECT_NAME} PRIVATE
35 # Add user sources here
36 )
37 # Add include paths
38 target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
39 # Add user defined include paths
40 )
41 # Add project symbols (macros)
42 target_compile_definitions(${CMAKE_PROJECT_NAME} PRIVATE
43 # Add user defined symbols
44 )
45 # Remove wrong libob.a library dependency when using cpp files

```

```

46     list(REMOVE_ITEM CMAKE_C_IMPLICIT_LINK_LIBRARIES ob)
47     # Add linked libraries
48     target_link_libraries(${CMAKE_PROJECT_NAME}
49         stm32cubemx
50         # Add user defined libraries
51     )

```

4.2 Modificación de Fuentes y Cabeceras (Después)

Para agregar la biblioteca del sensor de luz OPT3001, se debe editar el archivo CMakeLists.txt en dos bloques específicos:

1. **target_sources**: Se debe listar el archivo de implementación del sensor (Core/OPT3001/Src/OPT3001.c) para que CMake lo compile.
2. **target_include_directories**: Se debe indicar la ruta de las cabeceras (Core/OPT3001/Inc) para que las directivas `#include` puedan localizar los archivos de cabecera `.h`.

Las modificaciones específicas en dichos bloques se presentan de la siguiente manera:

```

1     # Add sources to executable
2     target_sources(${CMAKE_PROJECT_NAME} PRIVATE
3         # Add user sources here
4         Core/OPT3001/Src/OPT3001.c
5     )
6     # Add include paths
7     target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
8         # Add user defined include paths
9         Core/OPT3001/Inc
10    )

```

Reconfiguración del Proyecto

Una vez que haya editado y guardado su archivo CMakeLists.txt, recuerde abrir la paleta de comandos de VS Code (`Ctrl` + `Shift` + `P`) y ejecutar la tarea **CMake: Configure**. Esto refrescará la base de datos de compilación y preparará la extensión para compilar los nuevos archivos del sensor sin inconvenientes.

5 CONFIGURACIÓN DEL PUERTO SERIE E IMPRESIÓN DE VALORES FLOTANTES

Para validar el correcto funcionamiento de sensores analógicos o digitales que entreguen magnitudes físicas con decimales (como el sensor de luz OPT3001), es necesario enviar las lecturas a través de la interfaz USART hacia una terminal en la computadora.

5.1 Código de Prueba para el Sensor OPT3001

El siguiente código fuente en C para el archivo `Core\Src\main.c` realiza la inicialización del sensor a través del bus I2C1, configura sus límites de lux inferior y superior, y transmite de forma periódica las lecturas del registro de configuración y los valores calculados de lux (en formato de punto flotante) a través de la interfaz USART2:

```

1  /* USER CODE BEGIN Header */
2  /**
3  ****
4  * @file           : main.c
5  * @brief          : Main program body
6  ****
7  * @attention
8  *
9  * Copyright (c) 2026 STMicroelectronics.
10 * All rights reserved.
11 *
12 * This software is licensed under terms that can be found in the LICENSE file
13 * in the root directory of this software component.
14 * If no LICENSE file comes with this software, it is provided AS-IS.
15 *
16 ****
17 */
18 /* USER CODE END Header */
19 /* Includes -----*/
20 #include "main.h"
21 #include "stm32f3xx_hal.h"
22
23 /* Private includes -----*/
24 /* USER CODE BEGIN Includes */
25 #include <string.h>
26 #include <stdio.h>
27 #include <OPT3001.h>
28 /* USER CODE END Includes */
29
30 /* Private typedef -----*/
31 /* USER CODE BEGIN PTD */
32 OPT3001_HandleTypeDef opt3001;
33 /* USER CODE END PTD */
34
35 /* Private define -----*/
36 /* USER CODE BEGIN PD */
37
38 /* USER CODE END PD */
39
40 /* Private macro -----*/
41 /* USER CODE BEGIN PM */
42
43 /* USER CODE END PM */
44
45 /* Private variables -----*/

```



```

46 I2C_HandleTypeDef hi2c1;
47
48 UART_HandleTypeDef huart2;
49
50 /* USER CODE BEGIN PV */
51 char msg[50];
52 static uint16_t config = 0;
53 static double lowLimitLux = 0.0;
54 static double highLimitLux = 0.0;
55 static double lowLimit = 100.0;
56 static double highLimit = 2000.0;
57 static double lux = 0.0;
58 /* USER CODE END PV */
59
60 /* Private function prototypes -----*/
61 void SystemClock_Config(void);
62 static void MX_GPIO_Init(void);
63 static void MX_USART2_UART_Init(void);
64 static void MX_I2C1_Init(void);
65 /* USER CODE BEGIN PFP */
66
67 /* USER CODE END PFP */
68
69 /* Private user code -----*/
70 /* USER CODE BEGIN 0 */
71
72 /* USER CODE END 0 */
73
74 /**
75  * @brief The application entry point.
76  * @retval int
77  */
78 int main(void)
79 {
80
81     /* USER CODE BEGIN 1 */
82
83     /* USER CODE END 1 */
84
85     /* MCU Configuration-----*/
86
87     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
88     HAL_Init();
89
90     /* USER CODE BEGIN Init */
91
92     /* USER CODE END Init */
93
94     /* Configure the system clock */
95     SystemClock_Config();
96
97     /* USER CODE BEGIN SysInit */
98
99     /* USER CODE END SysInit */
100
101     /* Initialize all configured peripherals */
102     MX_GPIO_Init();
103     MX_USART2_UART_Init();
104     MX_I2C1_Init();
105     /* USER CODE BEGIN 2 */

```

```

106     OPT3001_Init(&opt3001, &hi2c1);
107     OPT3001_SetLowLimit(&opt3001, lowLimit);
108     OPT3001_SetHighLimit(&opt3001, highLimit);
109     /* USER CODE END 2 */
110
111     /* Infinite loop */
112     /* USER CODE BEGIN WHILE */
113     while (1)
114     {
115         config = OPT3001_GetConfiguration(&opt3001);
116         sprintf(msg, "CONFIGURATION Register = 0x%04X \r\n", config);
117         HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
118
119         lowLimitLux = OPT3001_GetLowLimitLux(&opt3001);
120         sprintf(msg, "Low Limit Lux = %.2f \r\n", lowLimitLux);
121         HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
122
123         highLimitLux = OPT3001_GetHighLimitLux(&opt3001);
124         sprintf(msg, "High Limit Lux = %.2f \r\n", highLimitLux);
125         HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
126
127         lux = OPT3001_GetLux(&opt3001);
128         sprintf(msg, "Lux = %.2f \r\n", lux);
129         HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
130
131         HAL_Delay(1000);
132         /* USER CODE END WHILE */
133
134         /* USER CODE BEGIN 3 */
135     }
136     /* USER CODE END 3 */
137 }
138
139 /**
140  * @brief System Clock Configuration
141  * @retval None
142  */
143 void SystemClock_Config(void)
144 {
145     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
146     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
147     RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};
148
149     /** Initializes the RCC Oscillators according to the specified parameters
150      * in the RCC_OscInitTypeDef structure.
151      */
152     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
153     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
154     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
155     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
156     RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
157     RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
158     RCC_OscInitStruct.PLL.PREDIV = RCC_PREDIV_DIV1;
159     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
160     {
161         Error_Handler();
162     }
163
164     /** Initializes the CPU, AHB and APB buses clocks
165      */

```

```

166     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
167     |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
168     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
169     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
170     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
171     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
172
173     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
174     {
175         Error_Handler();
176     }
177     PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_USART2|RCC_PERIPHCLK_I2C1;
178     PeriphClkInit.Usart2ClockSelection = RCC_USART2CLKSOURCE_PCLK1;
179     PeriphClkInit.I2c1ClockSelection = RCC_I2C1CLKSOURCE_HSI;
180     if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
181     {
182         Error_Handler();
183     }
184 }
185
186 /**
187  * @brief I2C1 Initialization Function
188  * @param None
189  * @retval None
190  */
191 static void MX_I2C1_Init(void)
192 {
193     hi2c1.Instance = I2C1;
194     hi2c1.Init.Timing = 0x00201D2B;
195     hi2c1.Init.OwnAddress1 = 0;
196     hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
197     hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
198     hi2c1.Init.OwnAddress2 = 0;
199     hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
200     hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
201     hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
202     if (HAL_I2C_Init(&hi2c1) != HAL_OK)
203     {
204         Error_Handler();
205     }
206
207     /** Configure Analogue filter
208     */
209     if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
210     {
211         Error_Handler();
212     }
213
214     /** Configure Digital filter
215     */
216     if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
217     {
218         Error_Handler();
219     }
220 }
221
222 /**
223  * @brief USART2 Initialization Function
224  * @param None
225  * @retval None

```

```

226 */
227 static void MX_USART2_UART_Init(void)
228 {
229     huart2.Instance = USART2;
230     huart2.Init.BaudRate = 115200;
231     huart2.Init.WordLength = UART_WORDLENGTH_8B;
232     huart2.Init.StopBits = UART_STOPBITS_1;
233     huart2.Init.Parity = UART_PARITY_NONE;
234     huart2.Init.Mode = UART_MODE_TX_RX;
235     huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
236     huart2.Init.OverSampling = UART_OVERSAMPLING_16;
237     huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
238     huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
239     if (HAL_UART_Init(&huart2) != HAL_OK)
240     {
241         Error_Handler();
242     }
243 }
244
245 /**
246  * @brief GPIO Initialization Function
247  * @param None
248  * @retval None
249  */
250 static void MX_GPIO_Init(void)
251 {
252     GPIO_InitTypeDef GPIO_InitStruct = {0};
253
254     /* GPIO Ports Clock Enable */
255     __HAL_RCC_GPIOC_CLK_ENABLE();
256     __HAL_RCC_GPIOF_CLK_ENABLE();
257     __HAL_RCC_GPIOA_CLK_ENABLE();
258     __HAL_RCC_GPIOB_CLK_ENABLE();
259
260     /*Configure GPIO pin Output Level */
261     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
262
263     /*Configure GPIO pin : B1_Pin */
264     GPIO_InitStruct.Pin = B1_Pin;
265     GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
266     GPIO_InitStruct.Pull = GPIO_NOPULL;
267     HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
268
269     /*Configure GPIO pin : LD2_Pin */
270     GPIO_InitStruct.Pin = LD2_Pin;
271     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
272     GPIO_InitStruct.Pull = GPIO_NOPULL;
273     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
274     HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
275 }
276
277 void Error_Handler(void)
278 {
279     __disable_irq();
280     while (1)
281     {
282     }
283 }

```

5.2 Uso de la Extensión Serial Monitor en Visual Studio Code

Para leer la terminal serie de la tarjeta sin salir del editor ni depender de software de terceros (como PuTTY o RealTerm), se utiliza la extensión oficial de Microsoft llamada **Serial Monitor**.

Cuando abre una nueva terminal en VS Code, aparecerá una pestaña dedicada titulada **SERIAL MONITOR**. Para inicializar la escucha del puerto COM al que está conectada su tarjeta, seleccione los parámetros correctos (Baud Rate: 115200) y haga clic sobre el botón **Start Monitoring** (tal como se ilustra en la figura 15).

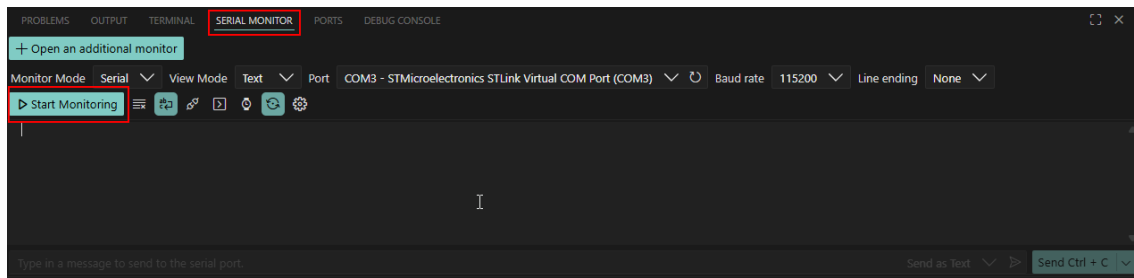


Figura 15: Consola e inicio de monitoreo en la extensión Serial Monitor.

5.3 El problema de la impresión de Punto Flotante

Una vez iniciado el monitoreo del puerto, observará que la consola imprime las cadenas, pero las variables decimales de tipo double o float se muestran completamente vacías en las llamadas de `printf`:

```
---- Opened the serial port COM3 ----
CONFIGURATION Register = 0xCCA4
Low Limit Lux =
High Limit Lux =
Lux =
```

Esto ocurre porque la librería estándar compacta de C para microcontroladores ARM GCC (*newlib-nano*) ****deshabilita por defecto el formateo de punto flotante**** en funciones como `printf` y `sprintf` para ahorrar memoria Flash en el chip.

5.4 Habilitación de Flotantes en el enlazador de CMake

Para habilitar el soporte de punto flotante, debemos modificar el archivo ****CMakeLists.txt**** principal de la raíz del proyecto para indicarle al enlazador del compilador GCC (*linker*) que cargue la biblioteca con soporte para flotantes usando la bandera ****`-u _printf_float`****.

Abra su archivo ****CMakeLists.txt**** y agregue la siguiente directiva de opciones de enlace (`target_link_options`) debajo de la declaración del proyecto:

```
1      # Habilitar el soporte de punto flotante para printf/sprintf (newlib-nano)
2      target_link_options(${CMAKE_PROJECT_NAME} PRIVATE
3  -u_printf_float
4  -u_scanf_float
5  )
```

Una vez agregado este comando al archivo, guarde los cambios, reconfigure CMake y compile nuevamente el proyecto. Al iniciar el monitor serie, verá los valores de luz decimales medidos por el sensor OPT3001 imprimiéndose correctamente.

⚠ ¿Por qué son necesarias estas banderas?

Los microcontroladores tienen memoria Flash muy limitada. Por ello, la biblioteca estándar de C utilizada en sistemas embebidos (newlib-nano) viene optimizada de fábrica:

- Por defecto, deshabilita el soporte para formatear números con decimales (**float** y **double**) en funciones de texto para ahorrar entre 10 KB y 15 KB de memoria Flash.
- Si intentas usar **%f** en un **sprintf** o **%lf** en un **sscanf** sin estas banderas, tu programa no podrá imprimir ni leer los decimales (se mostrarán vacíos o la función fallará).

El comportamiento y la función técnica de cada una de estas directivas de enlace se detalla a continuación:

- u **_printf_float**: Fuerza al enlazador (*linker*) a cargar el símbolo y la versión de las funciones de salida de texto (**printf**, **sprintf**, **snprintf**) compatibles con magnitudes decimales. Esto habilita el formateo correcto de especificadores como **%f**, **%e** y **%g**. Permite que puedas hacer **sprintf(msg, "Voltaje = %.2f V\r\n", 3.32);** y que se imprima "Voltaje = 3.32 V" en la consola serie en lugar de salir vacío.
- u **_scanf_float**: Fuerza al enlazador a inyectar el soporte de punto flotante en las funciones de análisis e ingreso de datos (**scanf**, **sscanf**). Esto permite que el sistema pueda interpretar y parsear magnitudes decimales representadas en formato de texto a variables reales en memoria (usando especificadores como **%f** o **%lf**). Permite que la línea **sscanf(rxBuffer, "SET:%d:%lf", &max_i, &shunt_r);** pueda extraer con éxito un valor decimal (como 0.00512 de resistencia shunt) a tu variable **double**. Si no tuvieras activada esta bandera, **sscanf** fallaría y nunca actualizaría la calibración del sensor.

5.5 Comparativa de Métodos para Habilitar Punto Flotante

Existen dos alternativas viables para indicarle al compilador GCC que incluya el soporte de punto flotante en las funciones de formateo (**printf/sprintf**). Ambos métodos logran el mismo resultado físico (inyectar la bandera **-u _printf_float** en el linker), pero difieren en el nivel de configuración:

MÉTODO A: MODIFICACIÓN EN LA TOOLCHAIN (**gcc-arm-none-eabi.cmake**)

Consiste en editar el archivo de configuración del compilador ubicado en **cmake▶gcc-arm-none-eabi.cmake**, modificando la definición de las banderas de enlace globales de la siguiente manera:

```

1 # Antes:
2 set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} --specs=nano.specs")
3
4 # Después:
5 set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} --specs=nano.specs -u
6     _printf_float")

```

- **Ventajas:** Mantiene el script de construcción principal **CMakeLists.txt** limpio de directivas de compilación específicas de la arquitectura de hardware.
- **Desventajas:** Si en el futuro realiza cambios de hardware en STM32CubeMX y regenera el proyecto, existe el riesgo de que los archivos de la carpeta **cmake▶** se sobreescriban y se pierda la configuración.

MÉTODO B: MODIFICACIÓN EN EL PROYECTO (**cmakelists.txt**)

Consiste en agregar la opción de enlace directamente sobre el objetivo ejecutable en el archivo principal de la raíz:

```
1 # Habilitar soporte de flotantes en el proyecto principal
2 target_link_options(${CMAKE_PROJECT_NAME} PRIVATE -u _printf_float)
3
```

- **Ventajas:** Sigue los estándares modernos de desarrollo en CMake (*target-centric*), es visible inmediatamente al abrir el proyecto y es completamente persistente ante cualquier regeneración de código de CubeMX.
- **Desventajas:** Mezcla banderas específicas del compilador GCC dentro del archivo de construcción principal.

i Recomendación Práctica

Aunque ambos métodos producen exactamente el mismo binario final, el **Método B** (modificar el archivo CMakeLists.txt) es el más recomendado para tus proyectos, ya que te garantiza que no perderás la visualización de decimales si necesitas volver a configurar pines o relojes en STM32CubeMX más adelante.

6 GESTIÓN DE MÚLTIPLES ARCHIVOS `main` EN UN SOLO PROYECTO

Durante la fase de prototipado o pruebas de un nuevo sensor (como el sensor de humedad y temperatura HDC1080), es común requerir la creación de diferentes scripts de prueba con lógicas e implementaciones distintas. Crear un proyecto nuevo desde cero en STM32CubeMX para cada prueba resulta ineficiente, consume excesivo almacenamiento en el disco duro y entorpece el flujo de trabajo.

Para evitar esto, podemos crear múltiples archivos principales en la misma carpeta del proyecto (por ejemplo: `main.c`, `main_1.c`, `main_2.c`). Sin embargo, el estándar de programación en C impone una restricción ineludible: **un ejecutable compilado solo puede poseer una única función `int main(void)`**. Si intentamos compilar el proyecto directamente con más de un archivo `main` activo, el enlazador (*linker*) detendrá el proceso arrojando el error crítico:

```
multiple definition of 'main'
multiple definition of 'Error_Handler'
```

A continuación, se exponen los diferentes métodos para gestionar esta estructura de forma de ordenar y excluir los archivos de prueba inactivos según el entorno utilizado.

6.1 Método 1: Exclusión de archivos en STM32CubeIDE

Si está programando bajo el entorno de desarrollo oficial STM32CubeIDE, el gestor de compilación interna basado en Eclipse permite excluir archivos individuales de la compilación de forma visual e intuitiva:

1. Agregue sus archivos secundarios a la carpeta **Core ▸ Src** con nombres descriptivos (ej. `main_1.c`).
2. En el panel del **Project Explorer** (lado izquierdo), haga clic derecho sobre el archivo que no desea compilar en su sesión de pruebas actual.
3. Desplácese a la opción **Resource Configurations** → **Exclude from Build...**
4. En la ventana que aparece, marque todas las configuraciones de compilación existentes (usualmente Debug y Release) y presione el botón **OK**.
5. Observará que el archivo excluido se torna de un color grisáceo y su icono presenta una franja diagonal. El compilador lo omitirá por completo en la siguiente compilación. Para activarlo nuevamente, realice el mismo procedimiento y desmarque las casillas de exclusión.

6.2 Método 2: Compilación Condicional en VS Code mediante CMake

En Visual Studio Code, cuando el proyecto utiliza un sistema de construcción guiado por CMake, el proceso de exclusión debe controlarse escribiendo una variable de selección condicional dentro del archivo **CMakeLists.txt** principal en la raíz del proyecto.

6.2.1 Paso A: Modificación en **CMakeLists.txt** principal

Abra el archivo **CMakeLists.txt** raíz y modifique la sección que define la creación del archivo ejecutable (`add_executable`) implementando la siguiente lógica:

```
1 # 1. Definir una variable de control para seleccionar el main activo
2 # Opciones validas: "MAIN" (para main.c), "TEST1" (para main_1.c), "TEST2" (para main_2.c)
3 set(ACTIVE_MAIN "TEST1")
4
5 # 2. Asignar de manera condicional la ruta del archivo fisico
6 if(ACTIVE_MAIN STREQUAL "MAIN")
7     set(MAIN_SOURCE "Core/Src/main.c")
8     message(STATUS "Compilando: MAIN PRINCIPAL (Core/Src/main.c)")
9 elseif(ACTIVE_MAIN STREQUAL "TEST1")
```



```

10  set(MAIN_SOURCE "Core/Src/main_1.c")
11  message(STATUS "Compilando: PRUEBA 1 (Core/Src/main_1.c)")
12  elseif(ACTIVE_MAIN STREQUAL "TEST2")
13      set(MAIN_SOURCE "Core/Src/main_2.c")
14      message(STATUS "Compilando: PRUEBA 2 (Core/Src/main_2.c)")
15  else()
16      message(FATAL_ERROR "ACTIVE_MAIN invalida. Usa 'MAIN', 'TEST1' o 'TEST2'.")
17  endif()
18
19  # 3. Crear el objeto ejecutable a partir de la fuente dinamica seleccionada
20  add_executable(${CMAKE_PROJECT_NAME}
21      ${MAIN_SOURCE}
22  )

```

6.2.2 Paso B: Modificación crítica en el script autogenerado de CubeMX

⚠ La trampa de stm32cubemx/CMakeLists.txt

Aunque modifique el archivo raíz anterior, la compilación de CMake volverá a fallar arrojando errores de duplicidad. Esto se debe a que la herramienta STM32CubeMX genera su propio subarchivo de configuración en `cmake\stm32cubemx\CMakeLists.txt`, en el cual agrega de forma fija e incondicional el archivo `main.c` a la variable `MX_Application_Src`:

```

set(MX_Application_Src
    ${CMAKE_CURRENT_SOURCE_DIR}/../Core/Src/main.c
    ${CMAKE_CURRENT_SOURCE_DIR}/../Core/Src/stm32f3xx_it.c
    ...

```

Para solucionarlo, debe abrir dicho subarchivo y **comentar o eliminar** la línea correspondiente al archivo principal:

```

set(MX_Application_Src
    # ${CMAKE_CURRENT_SOURCE_DIR}/../Core/Src/main.c # Excluido para control
    condicional
    ${CMAKE_CURRENT_SOURCE_DIR}/../Core/Src/stm32f3xx_it.c
    ...

```

Una vez completados estos dos pasos, para cambiar de una prueba a otra solo debe alterar el valor del string en la variable `ACTIVE_MAIN` de su `CMakeLists.txt` raíz, guardar y presionar **Build** (o **F7**) en VS Code.

6.3 Método 3: Truco Universal (Directivas del Preprocesador de C)

Este método representa la alternativa favorita de muchos desarrolladores, ya que es **completamente independiente del entorno de desarrollo (IDE) o del sistema de construcción** (Make, CMake, Ninja, etc.), y evita tener que realizar modificaciones manuales sobre configuraciones del proyecto o del compilador.

Consiste en crear un único archivo central de cabecera que defina cuál es la prueba activa, e inhabilitar condicionalmente el código de los archivos restantes utilizando el preprocesador de C.

1. **Creación de la cabecera de control:** Cree un archivo de configuración en la ruta `Core/Inc/test_config.h` con el siguiente contenido:

Listing 1: Archivo de configuración de pruebas test_config.h.

```

1 // Core/Inc/test_config.h
2 #ifndef TEST_CONFIG_H_
3 #define TEST_CONFIG_H_
4
5 // Descomenta unicamente la prueba que desees ejecutar:
6 #define RUN_MAIN_PRODUCTION
7 //#define RUN_TEST_1
8 //#define RUN_TEST_2
9
10 #endif
11

```

2. **Protección condicional de los archivos de código fuente:** En cada uno de sus archivos principales de pruebas, envuelva todo el contenido del archivo (incluyendo la declaración del 'main' y las funciones secundarias) dentro de una directiva condicional #ifdef:

- En Core/Src/main.c:

```

1 #include "test_config.h"
2 #ifdef RUN_MAIN_PRODUCTION
3
4 int main(void) {
5     // Código de producción principal...
6 }
7
8 #endif
9

```

- En Core/Src/main_test1.c:

```

1 #include "test_config.h"
2 #ifdef RUN_TEST_1
3
4 int main(void) {
5     // Código para probar el sensor HDC1080...
6 }
7
8 #endif
9

```

- En Core/Src/main_test2.c:

```

1 #include "test_config.h"
2 #ifdef RUN_TEST_2
3
4 int main(void) {
5     // Código para otra prueba...
6 }
7
8 #endif
9

```

¿Cómo funciona bajo el capó?

El compilador compilará físicamente todos los archivos `.c` listados en tu proyecto, pero el preprocesador de C evaluará las directivas antes de generar el código de máquina. Al estar activa únicamente una directiva en `test_config.h`, los archivos inactivos se procesarán como si estuvieran totalmente vacíos tras el paso del preprocesador. De este modo, solo existirá una única definición de la función `main()` real en tiempo de enlazado, previniendo el error de colisión.

6.4 El problema visual de IntelliSense en los archivos inactivos

Cuando este esquema se implementa con éxito, es normal experimentar un error de interfaz en Visual Studio Code. Al abrir en el editor los archivos que se encuentran inactivos en ese momento (ej. `main_2.c` si la variable de selección es `TEST1`), el sistema desplegará advertencias de inclusión y líneas onduladas de color rojo en las librerías, acompañadas del siguiente mensaje en los problemas:

```
#include errors detected. Consider updating your compile_commands.json or includePath.
cannot open source file "main.h"
cannot open source file "HDC1080.h"
```

6.4.1 ¿Por qué ocurre?

Este comportamiento se debe a que la extensión oficial C/C++ de VS Code utiliza la base de datos de compilación denominada `compile_commands.json` para conocer las rutas de inclusión del proyecto. Como CMake compila de forma selectiva, **los archivos inactivos se omiten de la base de datos**. Al no estar listados, IntelliSense no sabe qué rutas de hardware y drivers pertenecen a estos archivos y entra en conflicto, aunque el firmware compile sin problemas. Para eliminar estas falsas alertas visuales y habilitar el autocompletado en todas las pruebas a la vez, se presentan dos posibles alternativas:

ALTERNATIVA A: BÚSQUEDA RECURSIVA DE CABECERAS EN VS CODE

Para abrir el archivo de configuración de C/C++ en VS Code, presione la combinación de teclas `Ctrl+Shift+P` para desplegar la paleta de comandos y escriba: **C/C++: Edit Configurations (JSON)**. Esto abrirá o creará el archivo de configuración en la ruta `.vscode/c_cpp_properties.json`.

Si el archivo no cuenta con el arreglo de rutas de inclusión (lo cual es común en proyectos nuevos), se visualizará de la siguiente manera:

Listing 2: Archivo `c_cpp_properties.json` original (Sin `includePath`)

```
1 {
2   "configurations": [
3     {
4       "name": "STM32",
5       "compileCommands": "${workspaceFolder}/build/Debug/compile_commands.json",
6       "intelliSenseMode": "windows-msvc-x64",
7       "cStandard": "c17",
8       "cppStandard": "c++17"
9     }
10  ],
11  "version": 4
12 }
```

Para habilitar la indexación recursiva de cabeceras en todos los archivos, se debe modificar agregando el parámetro `"includePath"` y la regla recursiva (asegúrese de agregar la coma de separación en la propiedad anterior):

Listing 3: Archivo c_cpp_properties.json optimizado (Con includePath)

```

1 {
2   "configurations": [
3     {
4       "name": "STM32",
5       "compileCommands": "${workspaceFolder}/build/Debug/compile_commands.json",
6       "includePath": [
7         "${workspaceFolder}/**"
8       ],
9       "intelliSenseMode": "windows-msvc-x64",
10      "cStandard": "c17",
11      "cppStandard": "c++17"
12    }
13  ],
14  "version": 4
15 }

```

ALTERNATIVA B: CREACIÓN DE UN TARGET FICTICIO EN CMAKE

Agregue al final de su archivo CMakeLists.txt raíz una biblioteca ficticia agrupando todos los archivos de prueba inactivos, pero calificada con la directiva EXCLUDE_FROM_ALL para evitar que el compilador los enlace realmente en el archivo binario:

```

1 # Registrar las fuentes inactivas en CMake solo para que IntelliSense las indexe
2 add_library(inactive_mains EXCLUDE_FROM_ALL
3   Core/Src/main.c
4   Core/Src/main_1.c
5   Core/Src/main_2.c
6 )
7 target_link_libraries(inactive_mains PRIVATE stm32cubemx)
8 target_include_directories(inactive_mains PRIVATE Core/HDC1080)
9

```